Descripción de Base de Datos

Brenda Elizabeth Robles Concepcion
100067220

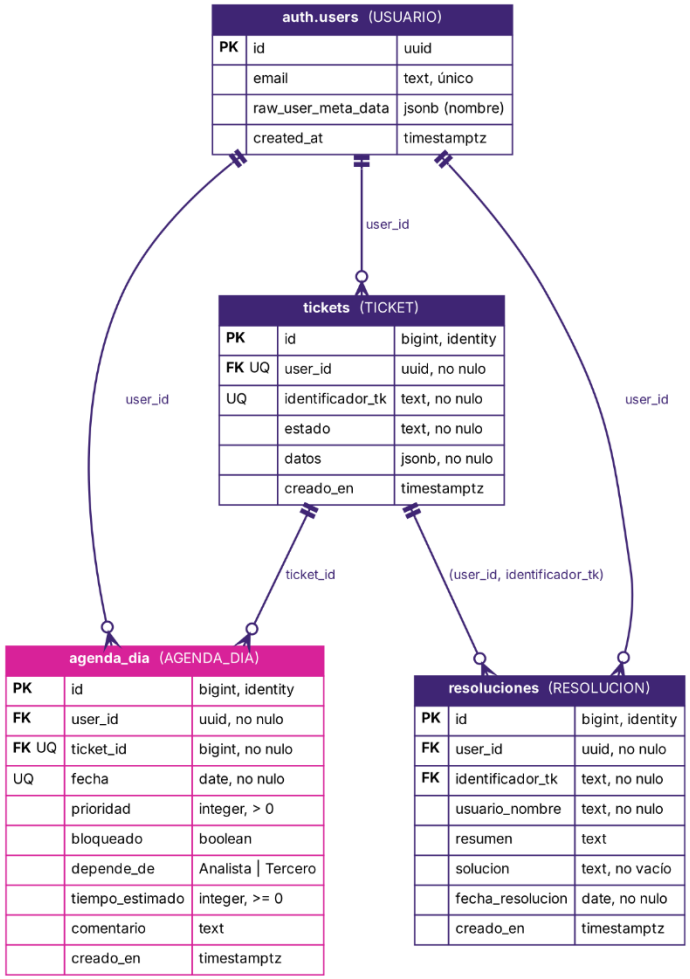
SEMINARIO DE PROYECTOS II
Henry Marcelino Candelario Rosario

1. Descripción general

La base de datos de GESTIONA! está implementada en PostgreSQL mediante Supabase y almacena la información necesaria para autenticar usuarios, conservar tickets importados desde Jira, organizar la agenda diaria y registrar las soluciones aplicadas. La estructura se diseñó para mantener separados los datos operativos del sistema y, al mismo tiempo, permitir que la información variable proveniente de los archivos Excel se conserve sin perder campos.

Entidades principales: auth.users (usuarios), tickets, agenda_dia y resoluciones. La tabla de usuarios es administrada directamente por Supabase Auth, las demás pertenecen al esquema público.

2. Modelo lógico



DICCIONARIO DE DATOS

A continuación se describen únicamente los campos utilizados por GESTIONA! y las reglas más importantes de cada entidad.

auth.users (Usuario)

Tabla administrada por Supabase Auth. Representa a la persona que inicia sesión y permite asociar los registros del sistema con su propietario.

Campo	Tipo	Llave / regla	Descripción
id	uuid	PK	Identificador único del usuario.
email	text	Único	Correo utilizado para autenticarse.
raw_user_meta_data	jsonb	-	Metadatos del usuario, como el nombre.
created_at	timestamptz	-	Fecha y hora de creación de la cuenta.

tickets (Ticket)

Almacena los requerimientos importados desde los archivos Excel de Jira. Cada ticket pertenece al usuario que lo cargó.

Campo	Tipo	Llave / regla	Descripción
id	bigint	PK / identity	Identificador interno autogenerado.
user_id	uuid	FK → auth.users	Usuario propietario del ticket.

Campo	Tipo	Llave / regla	Descripción
identificador_tk	text	No nulo / único por usuario	Clave del ticket en Jira.
estado	text	No nulo	Estado recibido desde el Excel.
datos	jsonb	No nulo	Fila completa del Excel, conserva los demás campos del ticket.
creado_en	timestamptz	Default now()	Fecha y hora en que se almacenó.

3.3 agenda_dia (Agenda diaria)

Registra qué tickets se trabajarán en una fecha determinada y conserva la información utilizada para su seguimiento.

Campo	Tipo	Llave / regla	Descripción
id	bigint	PK / identity	Identificador interno.
user_id	uuid	FK → auth.users	Propietario de la agenda.
ticket_id	bigint	FK → tickets	Ticket programado.
fecha	date	No nulo	Día correspondiente a la agenda.
prioridad	integer	> 0	Orden de trabajo, 1 representa el primero.
bloqueado	boolean	Default false	Indica si el ticket está bloqueado.
depende_de	text	Analista / Tercero	Indica de quién depende el desbloqueo.
tiempo_estimado	integer	>= 0	Minutos estimados para trabajar el ticket.
comentario	text	Opcional	Notas relacionadas con el seguimiento.
creado_en	timestamptz	Default now()	Fecha y hora de creación del registro.

HISTORIAL DE SOLUCIONES

Conserva el conocimiento generado al resolver un ticket. Permite saber quién lo resolvió, cuándo y cuál fue la solución aplicada.

Campo	Tipo	Llave / regla	Descripción
id	bigint	PK / identity	Identificador interno.
user_id	uuid	FK → auth.users	Usuario que registró la solución.
usuario_nombre	text	No nulo	Nombre visible de quien resolvió.
identificador_tk	text	FK compuesta con user_id	Clave del ticket resuelto.
resumen	text	Opcional	Título o resumen del ticket.
solucion	text	No nulo / no vacío	Descripción de cómo se resolvió.
fecha_resolucion	date	No nulo	Día en que se resolvió el ticket.
creado_en	timestamptz	Default now()	Fecha y hora del registro.

RELACIONES E INTEGRIDAD

Un usuario puede tener varios tickets, registros de agenda y resoluciones mediante el campo user_id.

agenda_dia se relaciona con tickets mediante ticket_id. Un mismo ticket no puede repetirse dos veces en la misma fecha.

resoluciones se vincula con el ticket mediante la combinación user_id + identificador_tk, evitando registrar soluciones para tickets inexistentes.

El identificador del ticket debe contener un valor y es único por usuario, por esto dos usuarios pueden tener la misma clave de Jira sin conflicto.

La prioridad debe ser mayor que 0, el tiempo estimado no puede ser negativo y depende_de solo admite Analista o Tercero.

La solución de un ticket es obligatoria y no puede almacenarse vacía.

SEGURIDAD

La base utiliza Row Level Security (RLS). Cada usuario autenticado puede consultar y modificar únicamente sus tickets y su agenda. El historial de resoluciones es visible para los usuarios autenticados como conocimiento compartido del equipo, pero cada usuario solo puede crear, editar o eliminar sus propias resoluciones.

NORMALIZACIÓN Y MANEJO DEL EXCEL

La información principal está separada en entidades con responsabilidades distintas: tickets conserva el requerimiento, agenda_dia la planificación y resoluciones el conocimiento generado. El campo datos de tickets utiliza JSONB de forma intencional para guardar la fila completa del Excel, ya que las columnas exportadas pueden variar. Los campos necesarios para relaciones, seguridad y consultas frecuentes se mantienen de forma estructurada.

CODIGO

TABLAS

-- TICKET: incidencia subida desde el Excel de Jira, a nombre de su dueño

```
create table if not exists public.tickets (
  id              bigint generated always as identity primary key,
  user_id         uuid       not null default auth.uid()
                             references auth.users (id) on delete cascade,
  identificador_tk  text       not null,
  estado          text       not null default 'Sin dato',
  datos           jsonb      not null default '{}'::jsonb,
  creado_en       timestamptz not null default now(),
  constraint uq_tickets_usuario_tk   unique (user_id, identificador_tk),
  constraint ck_tickets_identificador check (length(trim(identificador_tk)) > 0)
);
```

-- AGENDA_DIA: ticket programado para trabajarse en una fecha

```
create table if not exists public.agenda_dia (
  id              bigint generated always as identity primary key,
  user_id         uuid    not null default auth.uid()
```

```
                    references auth.users (id) on delete cascade,
  ticket_id       bigint  not null
                    references public.tickets (id) on delete cascade,
  fecha           date    not null,
  prioridad       integer not null,
  bloqueado       boolean not null default false,
  depende_de      text,
  tiempo_estimado  integer,
  comentario      text,
  creado_en       timestamptz not null default now(),
  constraint uq_agenda_fecha_ticket unique (fecha, ticket_id),
  constraint ck_agenda_prioridad    check (prioridad > 0),
  constraint ck_agenda_depende_de   check (depende_de is null or depende_de in ('Analista', 'Tercero')),
  constraint ck_agenda_tiempo       check (tiempo_estimado is null or tiempo_estimado >= 0)
);
```

-- RESOLUCION: historial de cómo se resolvió cada ticket (conocimiento del equipo)

```
create table if not exists public.resoluciones (
  id               bigint generated always as identity primary key,
  user_id          uuid  not null default auth.uid()
                     references auth.users (id) on delete cascade,
  usuario_nombre    text  not null,
  identificador_tk  text  not null,
  resumen           text,
  solucion          text  not null,
  fecha_resolucion  date  not null,
  creado_en         timestamptz not null default now(),
  constraint ck_resoluciones_solucion check (length(trim(solucion)) > 0),
  constraint fk_resoluciones_ticket
    foreign key (user_id, identificador_tk)
    references public.tickets (user_id, identificador_tk)
);
```

ÍNDICES (aceleran las consultas más frecuentes de la aplicación)

```
create index if not exists ix_tickets_usuario_estado on public.tickets (user_id, estado);
```

```
create index if not exists ix_agenda_usuario_fecha   on public.agenda_dia (user_id, fecha);
create index if not exists ix_resol_fecha            on public.resoluciones (fecha_resolucion desc);
create index if not exists ix_resol_usuario          on public.resoluciones (user_id);
create index if not exists ix_resol_ticket           on public.resoluciones (user_id, identificador_tk);
```

SEGURIDAD A NIVEL DE FILA (Row Level Security)

```
--    Tickets y agenda: cada usuario ve y modifica solo lo suyo.
--    Resoluciones: todo el equipo las lee; solo se crea/edita/borra lo propio.
alter table public.tickets      enable row level security;
alter table public.agenda_dia   enable row level security;
alter table public.resoluciones enable row level security;

drop policy if exists "tickets: solo los del usuario" on public.tickets;
create policy "tickets: solo los del usuario" on public.tickets
  for all to authenticated
  using (user_id = auth.uid()) with check (user_id = auth.uid());
```

```
drop policy if exists "agenda: solo la del usuario" on public.agenda_dia;
create policy "agenda: solo la del usuario" on public.agenda_dia
  for all to authenticated
  using (user_id = auth.uid()) with check (user_id = auth.uid());

drop policy if exists "resoluciones: lectura del equipo" on public.resoluciones;
create policy "resoluciones: lectura del equipo" on public.resoluciones
  for select to authenticated using (true);
drop policy if exists "resoluciones: crear las propias" on public.resoluciones;
create policy "resoluciones: crear las propias" on public.resoluciones
  for insert to authenticated with check (user_id = auth.uid());
drop policy if exists "resoluciones: editar las propias" on public.resoluciones;
create policy "resoluciones: editar las propias" on public.resoluciones
  for update to authenticated
  using (user_id = auth.uid()) with check (user_id = auth.uid());
drop policy if exists "resoluciones: borrar las propias" on public.resoluciones;
create policy "resoluciones: borrar las propias" on public.resoluciones
  for delete to authenticated using (user_id = auth.uid());
```

Conclusión

La base de datos de GESTIONA! permite mantener organizada y relacionada la información necesaria para la gestión diaria de tickets. Su diseño garantiza la asociación de los registros con cada usuario, controla datos inválidos mediante restricciones y protege la información con políticas RLS. Además, el historial de resoluciones permite conservar las soluciones aplicadas como conocimiento reutilizable dentro del sistema.